Song ID Hashing – C Program and Output

Part A, B and C: Division Method Hashing with Linear Probing, Search Comparison, Collisions and Load Factor.

C PROGRAM

```
#include <stdio.h>

#define SIZE 10
#define N 8

int hashTable[SIZE];

int hashFunction(int key)
{
    return key % SIZE;
}

void insert(int key)
{
    int index = hashFunction(key);
    int originalIndex = index;

    while (hashTable[index] != -1)
    {
        printf("Collision for %d at index %d\n", key, index);
        index = (index + 1) % SIZE;

        if (index == originalIndex)
        {
            printf("Hash table is full!\n");
            return;
        }
    }

    hashTable[index] = key;
    printf("Inserted %d at index %d\n", key, index);
}

void display()
{
    int i;

    printf("\nHash Table:\n");

    for (i = 0; i < SIZE; i++)
    {
        if (hashTable[i] == -1)
            printf("Index %d : EMPTY\n", i);
        else
            printf("Index %d : %d\n", i, hashTable[i]);
    }
}

int hashSearch(int key, int *operations)
{
    int index = hashFunction(key);
    int start = index;

    *operations = 0;

    while (hashTable[index] != -1)
    {
        (*operations)++;

        if (hashTable[index] == key)
            return index;

        index = (index + 1) % SIZE;

        if (index == start)
            break;
    }

    return -1;
}

int linearSearch(int arr[], int n, int key, int *comparisons)
{
    int i;
    *comparisons = 0;
```

```
    for (i = 0; i < n; i++)
    {
        (*comparisons)++;

        if (arr[i] == key)
            return i;
    }

    return -1;
}

int main()
{
    int songs[N] = {
        105, 210, 315, 420,
        525, 630, 735, 840
    };

    int i, key;
    int hashResult, linearResult;
    int hashOperations, linearComparisons;

    for (i = 0; i < SIZE; i++)
        hashTable[i] = -1;

    printf("----- INSERTION -----\n");

    for (i = 0; i < N; i++)
    {
        insert(songs[i]);
        display();
        printf("\n");
    }

    printf("\n----- FINAL HASH TABLE -----\n");
    display();

    printf("\n----- SEARCH -----\n");
    printf("Enter Song ID to search: ");
    scanf("%d", &key);

    hashResult = hashSearch(key, &hashOperations);

    linearResult = linearSearch(
        songs, N, key, &linearComparisons
    );

    if (hashResult != -1)
        printf("\nHashing: %d found at index %d\n",
               key, hashResult);
    else
        printf("\nHashing: %d not found\n", key);

    printf("Hashing operations = %d\n", hashOperations);

    if (linearResult != -1)
        printf("\nLinear Search: %d found at position %d\n",
               key, linearResult + 1);
    else
        printf("\nLinear Search: %d not found\n", key);

    printf("Linear Search comparisons = %d\n",
           linearComparisons);

    printf("\n----- ANALYSIS -----\n");

    printf("Number of elements = %d\n", N);
    printf("Hash table size = %d\n", SIZE);
    printf("Load Factor = %d / %d = %.2f\n",
           N, SIZE, (float)N / SIZE);

    printf("\nAverage Case Hashing Complexity: O(1)\n");
    printf("Worst Case Hashing Complexity: O(n)\n");
    printf("Linear Search Complexity: O(n)\n");

    return 0;
}
```

SAMPLE OUTPUT

Input used for search: 735

```
----- INSERTION -----
Inserted 105 at index 5
Index 0 : EMPTY
Index 1 : EMPTY
Index 2 : EMPTY
Index 3 : EMPTY
Index 4 : EMPTY
Index 5 : 105
Index 6 : EMPTY
Index 7 : EMPTY
Index 8 : EMPTY
Index 9 : EMPTY

Inserted 210 at index 0
Index 0 : 210
Index 1 : EMPTY
Index 2 : EMPTY
Index 3 : EMPTY
Index 4 : EMPTY
Index 5 : 105
Index 6 : EMPTY
Index 7 : EMPTY
Index 8 : EMPTY
Index 9 : EMPTY

Collision for 315 at index 5
Inserted 315 at index 6

Collision for 420 at index 0
Inserted 420 at index 1

Collision for 525 at index 5
Collision for 525 at index 6
Inserted 525 at index 7

Collision for 630 at index 0
Collision for 630 at index 1
Inserted 630 at index 2

Collision for 735 at index 5
Collision for 735 at index 6
Collision for 735 at index 7
Inserted 735 at index 8

Collision for 840 at index 0
Collision for 840 at index 1
Collision for 840 at index 2
Inserted 840 at index 3

----- FINAL HASH TABLE -----
Index 0 : 210
Index 1 : 420
Index 2 : 630
Index 3 : 840
Index 4 : EMPTY
Index 5 : 105
Index 6 : 315
Index 7 : 525
Index 8 : 735
Index 9 : EMPTY

----- SEARCH -----
Enter Song ID to search: 735

Hashing: 735 found at index 8
Hashing operations = 4

Linear Search: 735 found at position 7
Linear Search comparisons = 7

----- ANALYSIS -----
Number of elements = 8
Hash table size = 10
Load Factor = 8 / 10 = 0.80

Average Case Hashing Complexity: O(1)
Worst Case Hashing Complexity: O(n)
Linear Search Complexity: O(n)
```